

AI Code Review & Security Analysis Report

Professional Code Quality & Security Assessment

Generated: 2026-08-14 09:29 UTC | Language: python | Model: llama-3.3-70b-versatile

Executive Summary

Code Health Score	Risk Score	Total Findings
0/100	45	12

Merge Readiness: **Not Ready**

The code review has identified 12 findings with a risk score of 45, indicating a significant security risk. The majority of the findings are critical, highlighting the need for immediate attention to prevent potential security breaches.

Severity Breakdown

Critical	High	Medium	Low
9	3	0	0

Detailed Findings

Type	Severity	Line	Message	Recommendation
1. Hardcoded Password	● High	1	password = "admin123"	Store passwords in environment variables.
2. Hardcoded API Key	● High	3	api_key = "123456789"	Store secrets securely.
3. SQL Injection	● Critical	7	query = "SELECT * FROM users WHERE name='" + name + "'"	Use parameterized queries.
4. Dangerous Dynamic Execution	● Critical	9	eval(input())	Avoid dynamic code execution.
5. Dangerous Dynamic Execution	● Critical	11	exec("print('Hello')")	Avoid dynamic code execution.

Type	Severity	Line	Message	Recommendation
6. Command Injection	● Critical	15	subprocess.run("dir", shell=True)	Do not use shell=True with user input.
7. Security	● Critical	1	Hardcoded password	Use environment variables or a secure secrets management system to store sensitive information
8. Security	● Critical	2	Hardcoded API key	Use environment variables or a secure secrets management system to store sensitive information
9. Security	● Critical	5	SQL injection vulnerability	Use parameterized queries or prepared statements to prevent SQL injection
10. Security	● Critical	6	Use of eval() function with user input	Avoid using eval() with user input, consider using a safer alternative such as ast.literal_eval() or a parsing library
11. Security	● Critical	7	Use of exec() function	Avoid using exec() whenever possible, consider using a safer alternative such as a templating engine or a parsing library
12. Security	● High	10	Use of subprocess with shell=True	Avoid using shell=True whenever possible, consider using a safer alternative such as subprocess.run() with the capture_output argument

■ Remediation Roadmap

Priority-based action items to improve code quality and security

Prior ity	Finding Type	Severity	Impact	Suggested Action
#1	SQL Injection	● Critical	High	Replace the query with a parameterized query using a library like sqlite3 or psycopg2
#2	Dangerous Dynamic Execution	● Critical	High	Replace eval(input()) with a safer alternative, such as a dictionary or conditional statements, to handle user input.
#3	Command Injection	● Critical	High	Replace shell=True with shell=False and pass the command as a list of arguments
#4	Security	● Critical	High	Replace hardcoded passwords and API keys with environment variables or a secure secrets management system, and refactor code to avoid SQL injection, code injection, and command injection vulnerabilities
#5	Security	● Critical	High	Replace hardcoded API key with an environment variable
#6	Security	● Critical	High	Use parameterized queries or prepared statements to prevent SQL injection
#7	Security	● Critical	High	Avoid using eval() with user input, consider using a safer alternative such as ast.literal_eval() or a parsing library
#8	Security	● Critical	High	Avoid using exec() whenever possible, consider using a safer alternative such as a templating engine or a parsing library
#9	Security	● Critical	High	Replace exec() with a safer alternative and avoid using eval() and subprocess.run() with shell=True
#10	Hardcoded Password	● High	Medium	Store passwords in environment variables.
#11	Hardcoded API Key	● High	Medium	Store secrets securely.
#12	Security	● High	Medium	Avoid using subprocess with shell=True whenever possible, consider using a safer alternative such as subprocess.run() with the capture_output argument

■ Key Recommendations

1. Address all Critical and High severity findings before merging.
2. Run additional security scanning tools to complement this analysis.
3. Consider implementing automated testing to catch similar issues in the future.

■ Code Quality Metrics

Overall Health Score: 0/100

Risk Level: Unknown

Maintainability: N/A

Security Score: N/A

■ Source Code Snapshot

```
password = "admin123"
```

```
api_key = "123456789"
```

```
name = input()
```

```
query = "SELECT * FROM users WHERE name='" + name + "'"
```

```
eval(input())
```

```
exec("print('Hello')")
```

```
import subprocess
```

```
subprocess.run("dir", shell=True)
```



This report was generated by the AI Code Review & Security Analysis Agent. Review all findings carefully before implementing changes.

Report generated on 2026-08-14 09:29 UTC